University of Minho

Department of Informatics

Data Security, 26/27

REPORT

Assignment 04

AUTHOR

PG64892 **Marco Sèvegrand**

Submitted to/on

Blackboard

Date

October 30, 2026

Table of Contents

I Q1: AES-128-CTR 2
I.1 Method 2
I.2 Code 2
I.3 Result 2
II Q2: AES-128-ECB 3
II.1 Method 3
II.2 Code 3
II.3 Result 3
III Q3: AES-128-CBC with a predictable IV 4
III.1 Method 4
III.2 Code 4
III.3 Result 4
IV Q4: AES-128-GCM 5
IV.1 Method 5
IV.2 Code 5
IV.3 Result 5

I Q1: AES-128-CTR

I.1 Method

AES-CTR encrypts the counter block to produce a keystream and XORs it with the plaintext ($C = P \oplus K_s$). Because there is no integrity check, flipping bits in the ciphertext flips exactly the same bits in the decrypted plaintext.

The command ends with `NAOH_PPM=00100;`. The two digits to change from `0` to `1` are at byte positions `-6` and `-5` (the two leading digits of the value). Since `'0' xor '1'` is `0x30 xor 0x31 = 0x01`, I XORed the bytes at positions `-6` and `-5` of the ciphertext with `0x01` (`0xef xor 0x01 = 0xee` and `0xdc xor 0x01 = 0xdd`).

I.2 Code

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

nonce = bytes.fromhex("00000001a5f3c90e7b14d2660048ff21")
ct = bytearray(bytes.fromhex(
    "51073653f3851c51af06c462d293862e93941666f8d561dbefdcc89cd972"
))
key = bytes.fromhex("8b2d1f0a77c4e5519903aab4cde61277")  # verification only

# Flip "00" to "11" at byte positions -6 and -5 (the two leading value digits).
ct[-6] ^= ord("0") ^ ord("1")
ct[-5] ^= ord("0") ^ ord("1")
new_ct = bytes(ct)
print("New ciphertext:", new_ct.hex())

# Verification using the key (not part of the attack).
cipher = Cipher(algorithms.AES(key), modes.CTR(nonce))
decryptor = cipher.decryptor()
print("Decrypted:", (decryptor.update(new_ct) + decryptor.finalize()).decode())
```

I.3 Result

Q1.a

The forged ciphertext is:

`0x51073653f3851c51af06c462d293862e93941666f8d561dbeeddc89cd972`

Verified with the provided key, the original ciphertext decrypts to:

`PLC=07;CMD=SET;NAOH_PPM=00100;`

and the forged one to

`PLC=07;CMD=SET;NAOH_PPM=11100;`

Q1.b

Moving from ChaCha20 to AES-128-CTR does not protect against this attack. AES-CTR turns the block cipher into a stream cipher by XORing the generated keystream with the plaintext, and it uses no MAC or authentication tag. The ciphertext stays completely malleable, so an attacker who knows the fixed command format can flip specific bits exactly as with ChaCha20.

II Q2: AES-128-ECB

II.1 Method

In ECB mode each 16-byte plaintext block is encrypted independently with the same key ($C_i = E_{K(P_i)}$), so identical plaintext blocks always produce identical ciphertext blocks. To find identical commands, I compared every pair of full ciphertexts, then split them into 16-byte blocks (32 hex characters) to check for repeated blocks.

II.2 Code

```python
cts = {
    "C1": "b3f54ffd243d5764117c2751a130813068e1caf62ab7e794b90583176eeb9fc6"
          "542b191dd3114c5cdf9f06c11bbc31e0a2914503499340c978b69b612fcd6a6a",
    "C2": "a22cbbd3231e96c2e1300049d5ca045abbd343ee4ac57cbb1110f709dfeddd19"
          "143bf488dccb3bcf42644f665d08c99f57d3dfc3b1d11f4f952474ae36f72917",
    "C3": "b3f54ffd243d5764117c2751a130813068e1caf62ab7e794b90583176eeb9fc6"
          "542b191dd3114c5cdf9f06c11bbc31e0a2914503499340c978b69b612fcd6a6a",
    "C4": "456437c5194fde6cbcd694083dec9478d26e0d34661e257da479f73c4952885c"
          "577dd28d11e6d4c26e7de22affd8b8dbbb3b4fd998882afc390432cfb1e99781",
}

# Compare every pair of full ciphertexts.
names = list(cts)
for i, name in enumerate(names):
    for other in names[i + 1:]:
        result = "identical" if cts[name] == cts[other] else "different"
        print(f"{name} and {other}: {result}")

# Split into 16-byte (32 hex chars) blocks to see block-level leaks.
for name, ct in cts.items():
    blocks = [ct[i:i + 32] for i in range(0, len(ct), 32)]
    print(f"{name} blocks: {blocks}")
```

II.3 Result

Identical commands: Ciphertext 1 and Ciphertext 3 are completely identical, meaning they encrypt exactly the same command. Ciphertext 2 and Ciphertext 4 are different from each other and from Ciphertexts 1 and 3.

What AES-128-ECB leaks: it is deterministic and stateless, so it leaks equality of plaintext blocks. Anyone observing the traffic can tell when two messages are identical, or when different messages share identical 16-byte sections, without knowing the key.

Why ECB is unsuitable for PLC traffic: the commands follow a fixed format, so repeated commands or complete 16-byte blocks produce repeated ciphertext. For example, commands for PLC `07` have the same first block, `PLC=07;CMD=SET;N`, even when the concentration changes. This lets an attacker recognise repeated commands and learn patterns in the traffic without the key. Replay attacks are also possible if there is no separate replay protection, but this is not specific to ECB.

III Q3: AES-128-CBC with a predictable IV

III.1 Method

In CBC mode the first ciphertext block is $C_0 = E_{K(P_0 \oplus \mathrm{IV}_0)}$. For each query, the next IV is known, so I use it to prepare a chosen plaintext block:

$$P_{\text{injected}} = G \oplus \mathrm{IV}_0 \oplus \mathrm{IV}_{\text{next}} \tag{1}$$

where G is a guess for the secret first block P_0. When the control centre encrypts this block, the input to AES is:

$$P_{\text{injected}} \oplus \mathrm{IV}_{\text{next}} = G \oplus \mathrm{IV}_0 \tag{2}$$

If the guess matches P_0, the first ciphertext block equals c_0. The first plaintext block is `PLC=<id>;CMD=SET;N`, so I try ids from `00` to `99`. Both the oracle and the attack advance the IV counter after each query, starting at `iv1`.

III.2 Code

python

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

iv0 = bytes.fromhex("00000000000000000000000000001000")
c0  = bytes.fromhex("8e00a77c9519e662e83144b473edb147")
iv1 = bytes.fromhex("00000000000000000000000000001001")
key = bytes.fromhex("c0ffee1122334455aabbccddeeff0011")  # oracle emulation only
oracle_counter = int.from_bytes(iv1, "big")

def encryption_oracle(chosen_plaintext):
    global oracle_counter
    iv = oracle_counter.to_bytes(16, "big")
    oracle_counter += 1
    encryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).encryptor()
    return encryptor.update(chosen_plaintext) + encryptor.finalize()

# Predict each query's IV without using the key.
next_counter = int.from_bytes(iv1, "big")
for i in range(100):
    next_iv = next_counter.to_bytes(16, "big")
    guess = f"PLC={i:02d};CMD=SET;N".encode("ascii")
    injected_block = bytes(g ^ a ^ b for g, a, b in zip(guess, iv0, next_iv))
    response = encryption_oracle(injected_block)
    next_counter += 1
    if response[:16] == c0:
        print(f"Recovered <id>: {i:02d}")
        print(f"Full first block: {guess.decode()}")
        break
else:
    raise RuntimeError("No matching id found")
```

III.3 Result

Recovered id: `42` (full first block: `PLC=42;CMD=SET;N`).

Why a random IV prevents this attack: each guess needs the next IV before the chosen plaintext is submitted. A counter is unique but predictable, so the attacker can calculate it for every query. With a random, unpredictable IV, the attacker cannot prepare the XOR that cancels it out, so this comparison no longer tests the guess.

IV Q4: AES-128-GCM

IV.1 Method

I applied the same bit-flipping strategy as in Q1.a, XORing byte positions `-6` and `-5` of the ciphertext with `0x01` (`0x75 xor 0x01 = 0x74` and `0xdc xor 0x01 = 0xdd`), and then passed the modified ciphertext together with the original authentication tag to `AESGCM.decrypt` to see how the PLC reacts.

IV.2 Code

python

```python
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

nonce = bytes.fromhex("a0acc5b086bb58000c5b0d31")
ct = bytearray(bytes.fromhex(
    "8fc0cc2e6567e77bcfaad7c8363ae39b2db91d4df9aa5acb75dce650cfe6"
))
tag = bytes.fromhex("a903a1a5b290dfe9d41c2634deed9cb1")
key = bytes.fromhex("3a1c9f44db2e7780015566aabbcc9911")

# Flip "00" to "11" at byte positions -6 and -5.
ct[-6] ^= ord("0") ^ ord("1")
ct[-5] ^= ord("0") ^ ord("1")

# AES-GCM expects the tag appended to the ciphertext.
forged_data = bytes(ct) + tag

try:
    pt = AESGCM(key).decrypt(nonce, forged_data, None)
    print("Decrypted:", pt.decode())
except InvalidTag:
    print("Decryption failed: InvalidTag raised, forged message rejected.")
```

IV.3 Result

Q4.a

When the PLC decrypts and verifies the modified ciphertext, decryption fails with an `InvalidTag` exception and the message is discarded. AES-128-GCM defeats this attack because it checks an authentication tag as well as decrypting the message. Changing the ciphertext while keeping the original tag causes this check to fail, so the modified command is not accepted.

Q4.b

If the same `(key, nonce)` pair is reused for two different messages:

- **Confidentiality:** GCM uses CTR mode internally, so reusing the nonce produces the same keystream. For the overlapping bytes, XORing the ciphertexts gives the XOR of the plaintexts ($C_1 \oplus C_2 = P_1 \oplus P_2$). This does not reveal both messages on its own, but knowing part of one message reveals the same part of the other. The fixed PLC command format makes this especially useful to an attacker.
- **Authentication:** nonce reuse exposes a relationship between the tags that can help recover the secret value used to calculate them. Two messages may leave several possible values, but more observations can narrow them down and allow forged tags, so the PLC can no longer rely on the tag to reject modified commands.